B24 Convolutional networks

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B24 Convolutional networks 3
B24-L1 Convolution, a small filter slid across an image 3
B24-L2 Padding, stride, channels and the (N, C, H, W) layout 8
B24-L3 Pooling, flattening and a complete small CNN 13
B24-L4 Images as tensors, normalisation and augmentation 18
B24-L5 Transfer learning, a pretrained backbone with a new head 23
B24-L6 Evaluating image classifiers, per class and on honest splits 29
Track B best-practice checklist 35
Appendix: solutions and answer keys 36

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B24 Convolutional networks

Roadmap: Months 4 to 6: CNNs. Convolution from scratch, shapes, pooling, image tensors and augmentation, transfer learning and image-classifier evaluation.

B24-L1 Convolution, a small filter slid across an image

Concept

A dense layer connects every input to every output. Images need two things it lacks: nearby pixels belong together, and a pattern means the same wherever it appears. A **convolution** builds in both.

A small grid of weights, the **kernel** or **filter**, slides across the image. At each position, multiply the kernel by the patch under it, element by element, and sum. That gives one output value, and all the values together form a **feature map**:

```
def correlate(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = (image[i:i + kh, j:j + kw] * kernel).sum()
    return out
```

- **Local.** Each output sees only a `kh` by `kw` patch.
- **Shared.** The same weights slide everywhere, so an edge detector learned in one corner works in every corner. Sixteen 3 by 3 filters over a colour image hold 448 parameters, 16 × 3 × 3 × 3 weights plus 16 biases. A dense layer producing the same sixteen 30 by 30 maps from a 32 by 32 colour image would need over 44 million weights.
- **Learned.** You do not design the kernels. Training sets their weights, as in any layer (lesson B23-L3).

PyTorch's `nn.Conv2d` (PyTorch 2.15) computes this sliding sum, which mathematics calls **cross-correlation**: the kernel is not flipped. With the same weights copied in, `nn.Conv2d` matched the NumPy function above on a 4 by 4 test image.

An example: the kernel `[[-1, 1], [-1, 1]]` responds where brightness rises from left to right. On an image whose left half is 0 and right half is 1, its output is 2 along the boundary and 0 everywhere else. That is an edge

detector, and the first layer of a trained image model holds many filters like it.

Why it matters

A team classified scanned survey forms with a dense network on the flattened pixels. It scored well on the training scans and badly on forms scanned a few millimetres off-centre, because a tick box that moved by a few pixels landed on different weights. A small convolutional network, whose filters slide across the page, kept its accuracy on the shifted scans.

Worked example: weak code

```
import numpy as np


def correlate(image, kernel):
    out = np.zeros((image.shape[0] - 1, image.shape[1] - 1))
    for i in range(image.shape[0] - 1):
        for j in range(image.shape[1] - 1):
            out[i, j] = (image[i:i + 2, j:j + 2] * kernel).sum()
    return out
```

1. **Line 5.** Hardcoded for a 2 by 2 kernel. A 3 by 3 kernel gives wrong sizes, with no error.
2. **Line 8.** No check that the kernel fits, or that both inputs are 2-D.

Worked example: strong code

```
import numpy as np


def correlate2d(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Slide a kernel over an image without padding, as nn.Conv2d does.

    Args:
        image: 2-D array, shape (h, w).
        kernel: 2-D array, shape (kh, kw), no larger than the image.

    Returns:
        The feature map, shape (h - kh + 1, w - kw + 1).

    Raises:
        ValueError: If either input is not 2-D or the kernel is larger
    than the image.
    """
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError(f"need 2-D arrays, got {image.shape} and
    {kernel.shape}")
    kh, kw = kernel.shape
```

```
    out_h, out_w = image.shape[0] - kh + 1, image.shape[1] - kw + 1
    if out_h < 1 or out_w < 1:
        raise ValueError(f"kernel {kernel.shape} is larger than image {image.shape}")
    out = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = (image[i : i + kh, j : j + kw] * kernel).sum()
    return out
```

1. **Line 19.** Sizes come from the kernel, so any kernel works.
2. **Line 20.** One output per position where the whole kernel fits.
3. **Line 22.** Fail fast instead of returning an empty map.
4. **Line 26.** Multiply the patch by the kernel and sum: cross-correlation, no flip.

A convolution slides one small set of shared weights across the image. It is local, it works the same everywhere, and training learns its weights.

Recap card

- **What a convolution computes** At each position, the patch times the kernel, element by element, summed. The outputs form a feature map.
- **Output size without padding** (h - kh + 1, w - kw + 1).
- **Parameters in nn.Conv2d(in, out, k)** out * in * k * k weights plus out biases. Independent of image size.
- **Does nn.Conv2d flip the kernel?** No. It computes cross-correlation, the plain sliding sum.

Practice

Exercise 1. Multiple choice (level 1)

Why does a convolutional layer suit images better than a dense layer on flattened pixels?

- a) Each output looks at a small neighbourhood, and the same weights are reused at every position
- b) It has more parameters, so it can learn more
- c) It flips the kernel, which makes it rotation invariant
- d) It needs no training: the kernels are fixed edge detectors

Why?

- r1) Nearby pixels form patterns, and a pattern means the same wherever it appears. Local, shared weights encode both, with far fewer parameters.
- r2) Convolutions run faster than matrix multiplication on every device.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import numpy as np


def correlate(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = (image[i:i + kh, j:j + kw] * kernel).sum()
    return out


image = np.array([[0.0, 0.0, 1.0, 1.0]] * 4)
print(correlate(image, np.array([[-1.0, 1.0], [-1.0, 1.0]])).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete one output value of the sliding sum.

```
out[i, j] = (image[i:i + _____(1), j:j + kw] * kernel)._____(2)()
```

Exercise 4. Write the function (level 3)

Write `correlate2d(image, kernel) -> np.ndarray`, the sliding sum without padding.

- The output has shape `(h - kh + 1, w - kw + 1)`.
- Raise `ValueError` when either input is not 2-D, or the kernel is larger than the image in either direction.

Your code must pass these checks:

- Sum the patch times the kernel at every position where the kernel fits.
- Do not flip the kernel, and size the output from the kernel's own shape.
- Raise ValueError for inputs that are not 2-D, or a kernel larger than the image.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Slide a kernel over an image without padding, as nn.Conv2d does.
def correlate2d(
    image: np.ndarray,
    kernel: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np

```

```

def correlate2d(image, kernel):
    return image * kernel
```

Exercise 5. Write the function (level 3)

Write `conv_params(in_channels: int, out_channels: int, kernel_size: int) -> int`: the parameters in one square `nn.Conv2d` layer with a bias.

- Each output channel has one `kernel_size` by `kernel_size` weight grid per input channel, plus one bias.
- Raise `ValueError` when any argument is below 1.

Your code must pass these checks:

- Count out_channels * in_channels * kernel_size ** 2 weights plus out_channels biases.
- Raise ValueError when any argument is below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Parameters in a square Conv2d layer with bias.
def conv_params(
    in_channels: int,
    out_channels: int,
    kernel_size: int,
) -> int: ...
```

Starter code:

```
def conv_params(in_channels, out_channels, kernel_size):
    return in_channels * out_channels
```

Exercise 6. Spot the bug (level 4)

The feature maps come out one row and one column short, and edges at the bottom and right of every image are never detected. Tap the line responsible, then fix it.

```
import numpy as np


def correlate2d(image, kernel):
    kh, kw = kernel.shape
    out_h, out_w = image.shape[0] - kh, image.shape[1] - kw
    out = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = (image[i : i + kh, j : j + kw] * kernel).sum()
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: nn.Conv2d with copied weights equals the NumPy sliding sum (cross-correlation, no flip); Conv2d(3, 16, 3) has a (16, 3, 3, 3) weight and 448 parameters; the [[-1, 1], [-1, 1]] kernel gives 2 along a 0-to-1 vertical edge. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU) and NumPy, November 2026.)

B24-L2 Padding, stride, channels and the (N, C, H, W) layout

Concept

`nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)` (PyTorch 2.15) takes a batch shaped `(N, C, H, W)`: images, channels, height, width. Its weight has shape `(out_channels, in_channels, kH, kW)`.

The output height, and the width likewise, is `(H + 2 * padding - kernel_size) // stride + 1`. For a batch of 8 colour images of 32 by 32 and 16 filters:

- kernel 3: `(8, 16, 30, 30)`. Each layer without padding shrinks the map by `kernel_size - 1`.
- kernel 3, padding 1: `(8, 16, 32, 32)`. **Padding** adds a border of zeros, so edge pixels are covered as often as the rest, and the size stays. `padding="same"` does this for you, with stride 1 only: with a stride it raises "padding='same' is not supported for strided convolutions".
- kernel 3, stride 2, padding 1: `(8, 16, 16, 16)`. **Stride** 2 moves the kernel two pixels at a time and halves height and width.
- kernel 5, stride 2: `(8, 16, 14, 14)`, since `(32 - 5) // 2 + 1` is 14.

Channels. A colour image has 3 input channels. Each filter spans all of them and sums across them, giving one output channel. Sixteen filters give 16 channels, which the next layer reads as its input channels.

Layout. PyTorch puts channels first. NumPy arrays and image files usually hold `(N, H, W, C)`. Passing `(8, 32, 32, 3)` to `nn.Conv2d(3, ...)` raises "expected input[8, 32, 32, 3] to have 3 channels, but got 32 channels instead". Reorder with `x.permute(0, 3, 1, 2)`. `permute` reorders dimensions, like a transpose for any number of them. A single image of shape `(C, H, W)` also works, but training code passes batches.

Write the size formula down for each layer before you build a model. Lesson B24-L3 needs it to size the dense layer after the convolutions.

Why it matters

A first image model loaded photos with an imaging library into NumPy arrays of shape (N, 224, 224, 3) and passed them to `nn.Conv2d(3, 32, 3)`. The error named 224 channels, and the team spent an afternoon checking the photos for corruption. One `permute(0, 3, 1, 2)` moved the colour channels to where PyTorch expects them.

Worked example: weak code

```
1  import torch
2
3  batch = torch.as_tensor(images, dtype=torch.float32)
4  features = torch.nn.Conv2d(3, 16, 3, stride=2, padding="same")(batch)
```

1. **Line 3.** Images arrive as (N, H, W, C). Conv2d reads dimension 1 as channels.
2. **Line 4.** padding='same' with a stride raises a ValueError in PyTorch 2.15.

Worked example: strong code

```
1  import numpy as np
2  import torch
3  from torch import nn
4
5
6  def to_nchw(images: np.ndarray) -> torch.Tensor:
7      """Convert (N, H, W, C) images to a float32 (N, C, H, W) batch.
8
9      Raises:
10         ValueError: If the array is not 4-D.
11     """
12     if images.ndim != 4:
13         raise ValueError(f"expected (N, H, W, C), got {images.shape}")
14     return torch.as_tensor(images, dtype=torch.float32).permute(0, 3, 1,
       2)
15
16
17 def conv_output_size(size: int, kernel_size: int, stride: int = 1,
     padding: int = 0) -> int:
18     """Height or width after one Conv2d layer."""
19     return (size + 2 * padding - kernel_size) // stride + 1
20
21
22 DOWNSAMPLE = nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=1)
23 assert conv_output_size(32, 3, stride=2, padding=1) == 16
```

1. **Line 14.** Channels move to dimension 1, where Conv2d reads them.

2. **Line 19.** The size rule, written once and checked against each layer.
3. **Line 22.** Explicit padding 1 with stride 2 halves 32 to 16.

Conv2d reads (N, C, H, W). Output size is (H + 2p - k) // s + 1. Pad to keep the size, stride to shrink it, and permute images that arrive channels last.

Recap card

- **Conv2d output size** (H + 2 * padding - kernel_size) // stride + 1, for height and width alike.
- **Keep the size with a k by k kernel** Stride 1 and padding (k - 1) / 2, or padding='same' (stride 1 only).
- **Conv2d input layout** (N, C, H, W). Convert (N, H, W, C) with x.permute(0, 3, 1, 2), never reshape.
- **Conv2d weight shape** (out_channels, in_channels, kH, kW).

Practice

Exercise 1. Multiple choice (level 1)

Which layer keeps a 32 by 32 input at 32 by 32?

- a) `nn.Conv2d(3, 16, kernel_size=3, padding=1)`
- b) `nn.Conv2d(3, 16, kernel_size=3)`
- c) `nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=1)`
- d) `nn.Conv2d(3, 16, kernel_size=5, padding=1)`

Why?

- r1) (32 + 2 - 3) // 1 + 1 = 32: one pixel of padding on each side makes up for the kernel's overhang.
- r2) Padding always keeps the size, whatever the kernel.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
1 import torch
2 from torch import nn
3
4 x = torch.zeros(8, 3, 32, 32)
5 print(tuple(nn.Conv2d(3, 16, 3)(x).shape))
6 print(tuple(nn.Conv2d(3, 16, 3, stride=2, padding=1)(x).shape))
7 print(tuple(nn.Conv2d(3, 16, 5, stride=2)(x).shape))
```

Exercise 3. Fill the gap (level 2)

Complete the output-size rule, and the reorder from (N, H, W, C) to (N, C, H, W).

```
out = (size + 2 * padding - kernel_size) // _____(1) + 1
batch = images._____(2)(0, 3, 1, 2)
```

Exercise 4. Write the function (level 3)

Write `conv_output_size(size, kernel_size, stride=1, padding=0) -> int`.

- Return `(size + 2 * padding - kernel_size) // stride + 1`.
- Raise `ValueError` when `size`, `kernel_size` or `stride` is below 1, `padding` is negative, or the result would be below 1.

Your code must pass these checks:

- Return (size + 2 * padding - kernel_size) // stride + 1.
- Raise ValueError for sizes below 1, negative padding, or an output below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Height or width after one Conv2d or pooling layer.
def conv_output_size(
    size: int,
    kernel_size: int,
    stride: int = 1,
    padding: int = 0,
) -> int: ...
```

Starter code:

```
def conv_output_size(size, kernel_size, stride=1, padding=0):
    return size - kernel_size
```

Exercise 5. Write the function (level 3)

Write `stack_output_size(size: int, layers: list[tuple[int, int, int]]) -> int`: the height after a stack of layers, each given as `(kernel_size, stride, padding)`. `conv_output_size` from exercise 4 is defined for you, and raises `ValueError` when a layer does not fit.

- Apply the layers in order and return the final size.
- Return `size` unchanged for an empty list.

Your code must pass these checks:

- Apply conv_output_size layer by layer, in order.
- Apply the layers in the order given.

- Let conv_output_size's ValueError through when a layer no longer fits.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Height or width after a stack of conv or pooling layers.
def stack_output_size(
    size: int,
    layers: list[tuple[int, int, int]],
) -> int: ...
```

Starter code:

```python
def stack_output_size(size, layers):
    return size
```

Exercise 6. Spot the bug (level 4)

The first forward pass raises "expected input[64, 28, 28, 1] to have 1 channels, but got 28 channels instead". Tap the line responsible, then fix it.

```python
import numpy as np
import torch
from torch import nn

model = nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
    nn.Flatten(), nn.Linear(16 * 28 * 28, 10))


def to_batch(images: np.ndarray) -> torch.Tensor:
    return torch.as_tensor(images, dtype=torch.float32)


logits = model(to_batch(scans))
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: on (8, 3, 32, 32), Conv2d(3, 16, 3) gives (8, 16, 30, 30), with padding 1 (8, 16, 32, 32), with stride 2 and padding 1 (8, 16, 16, 16), kernel 5 stride 2 (8, 16, 14, 14); padding='same' with stride raises 'padding='same' is not supported for strided convolutions'; an (8, 32, 32, 3) input raises 'expected input[8, 32, 32, 3] to have 3 channels, but got 32 channels instead'. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)

B24-L3 Pooling, flattening and a complete small CNN

Concept

A CNN stacks convolution blocks, then a small classifier (PyTorch 2.15):

```
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 to 4x4
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 to 2x2
    nn.Flatten(),
    # (N, 32 * 2 * 2)
    nn.Linear(32 * 2 * 2, 10),
    # one logit per digit
)
```

Pooling shrinks each feature map. `nn.MaxPool2d(2)` keeps the largest value in each 2 by 2 block, moving 2 pixels at a time. A 32 by 32 map becomes 16 by 16, and an odd 5 by 5 map becomes 2 by 2: the last row and column are dropped. On `[[1, 3, 2, 0], [4, 2, 1, 5], [0, 1, 3, 2], [2, 6, 1, 1]]` it gives `[[4, 5], [6, 3]]`. `nn.AvgPool2d(2)` averages each block instead. Pooling has no weights. It makes features tolerate small shifts and cuts the work for later layers.

The usual pattern: early layers find edges, later layers combine them into parts, and each block halves the map while it doubles the channels.

Flatten, then size the dense layer. `nn.Flatten()` turns `(N, C, H, W)` into `(N, C * H * W)`. The `nn.Linear` after it must expect exactly that many inputs. Work it out with the size rule from lesson B24-L2: 8 by 8 digits after two blocks are 32 channels of 2 by 2, so 128. A wrong count fails on the first batch with "mat1 and mat2 shapes cannot be multiplied (64x128 and 2048x10)".

Global average pooling. `nn.AdaptiveAvgPool2d(1)` averages each channel to one value, whatever the input size: `(8, 64, 7, 5)` becomes `(8, 64, 1, 1)`. Ending with it, then `nn.Flatten()` and `nn.Linear(64, classes)`, gives a model that accepts any image size.

The digits model above has 6,090 parameters. A dense network with one hidden layer of 64 units has 4,810. The lab compares the two on the same data.

Why it matters

A model built for 32 by 32 product photos was reused on 64 by 64 photos. The Linear layer after Flatten expected 4,096 inputs and received 16,384, and the job failed on its first batch in production. Ending the convolutions with `nn.AdaptiveAvgPool2d(1)` made the classifier's input 64 for any photo size, and one model served both sizes.

Worked example: weak code

```
model = nn.Sequential(
    nn.Conv2d(1, 16, 3), nn.ReLU(),
    nn.Conv2d(16, 32, 3), nn.ReLU(),
    nn.Flatten(),
    nn.Linear(2048, 10),
)
```

1. **Line 2.** No padding and no pooling: the map only shrinks by 2 per layer, and the work stays high.
2. **Line 5.** A number copied from somewhere. Nothing ties it to the image size.

Worked example: strong code

```
from torch import nn


def make_cnn(channels: int, classes: int) -> nn.Sequential:
    """Two conv blocks, global average pooling and a linear classifier.

    Args:
        channels: Input channels, 1 for greyscale.
        classes: Number of classes, one logit each.

    Returns:
        A model that accepts images of any size of at least 4 by 4.
    """
    return nn.Sequential(
        nn.Conv2d(channels, 16, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Conv2d(16, 32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.AdaptiveAvgPool2d(1),
        nn.Flatten(),
        nn.Linear(32, classes),
    )
```

1. **Line 15.** Padding keeps the size, so only the MaxPool2d(2) layers shrink it, halving height and width with no weights.
2. **Line 21.** One value per channel, whatever the image size.

3. **Line 23.** Its input equals the last conv's channels, so it never needs recomputing.

Conv, ReLU and pool blocks shrink the maps and grow the channels. Size the dense layer from the shape rule, or end with global average pooling so the size never matters.

Recap card

- **What MaxPool2d(2) does** Largest value per non-overlapping 2 by 2 block. Halves height and width, rounding down. No weights.
- **Linear size after Flatten** channels * H * W of the last feature map. Work it out with the size rule, layer by layer.
- **Make a CNN accept any image size** End the convolutions with nn.AdaptiveAvgPool2d(1), then Flatten and Linear(channels, classes).
- **Typical CNN block** Conv2d (padding to keep the size), ReLU, MaxPool2d(2). Each block halves the map and grows the channels.

Practice

Exercise 1. Multiple choice (level 1)

What does `nn.MaxPool2d(2)` add to a CNN?

- a) It halves height and width with no weights, and makes features tolerate small shifts
- b) It learns which pixels matter most
- c) It doubles the number of channels
- d) It replaces the activation function

Why?

- r1) Keeping the largest value in each 2 by 2 block reports that a feature is present nearby, not its exact pixel, and quarters the work for later layers.
- r2) Pooling removes noise from the input images.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import torch
from torch import nn

m = torch.tensor([[1.0, 3.0, 2.0, 0.0],
                  [4.0, 2.0, 1.0, 5.0],
                  [0.0, 1.0, 3.0, 2.0],
                  [2.0, 6.0, 1.0, 1.0]]).reshape(1, 1, 4, 4)
```

```
print(nn.MaxPool2d(2)(m)[0, 0].tolist(), nn.AvgPool2d(2)(m)[0,
    0].tolist())
```

Exercise 3. Fill the gap (level 2)

Complete a block that halves the map, and the step before the classifier.

```
nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn._____(1)(2),
nn._____(2)(),
nn.Linear(32 * 2 * 2, 10),
```

Exercise 4. Write the function (level 3)

Write `max_pool2d(x, size=2) -> np.ndarray`: max pooling of a 2-D array with non-overlapping `size` by `size` blocks, as `nn.MaxPool2d(size)` does.

- The output has shape `(h // size, w // size)`. Leftover rows and columns are dropped.
- Raise `ValueError` when `x` is not 2-D, `size` is below 1, or `x` is smaller than one block.

Your code must pass these checks:

- Keep the largest value in each size by size block.
- Drop leftover rows and columns, and use size for both the block and the step.
- Raise ValueError for non-2-D input, size below 1, or an input smaller than one block.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Max pooling with non-overlapping size by size blocks.
def max_pool2d(x: np.ndarray, size: int = 2) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def max_pool2d(x, size=2):
    return x[::size, ::size]
```

Exercise 5. Write the function (level 3)

Write `flatten_features(image_size: int, blocks: int, channels: int) -> int`: the input size of the Linear layer after `blocks` repeats of "3 by 3 conv

with padding 1, ReLU, `MaxPool2d(2)`" on square images, where the last conv has `channels` filters.

- Each block keeps the size through the conv and halves it, rounding down, through the pool.
- Raise `ValueError` when `image_size` or `channels` is below 1, `blocks` is negative, or the map shrinks below 1.

Your code must pass these checks:

- Halve the size once per block, rounding down, then multiply channels by size squared.
- Raise ValueError for bad arguments or a map that shrinks below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Inputs to the Linear layer after conv, ReLU, MaxPool2d(2) blocks.
def flatten_features(
    image_size: int,
    blocks: int,
    channels: int,
) -> int: ...
```

Starter code:

```
def flatten_features(image_size, blocks, channels):
    return image_size * channels
```

Exercise 6. Spot the bug (level 4)

The first batch of 8 by 8 digits fails with "mat1 and mat2 shapes cannot be multiplied (64x128 and 2048x10)". Tap the line responsible, then fix it.

```
from torch import nn

model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(32 * 8 * 8, 10),
)
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: MaxPool2d(2) on the 4 by 4 example gives [[4, 5], [6, 3]] and AvgPool2d(2) [[2.5, 2.0], [2.25, 1.75]]; 5 by 5 pools to 2 by 2; AdaptiveAvgPool2d(1) maps (8, 64, 7, 5) to (8, 64, 1, 1); the two-block digits CNN has 6,090 parameters and the 64-unit dense network 4,810; a

Linear(2048, 10) after two pools on 8 by 8 raises 'mat1 and mat2 shapes cannot be multiplied (64x128 and 2048x10)'. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)

B24-L4 Images as tensors, normalisation and augmentation

Concept

Image files arrive as unsigned 8-bit integers from 0 to 255, usually laid out `(H, W, C)`. Before a model sees them (PyTorch 2.15, torchvision 0.29):

1. Move channels first: `torch.from_numpy(img).permute(2, 0, 1)` gives `(C, H, W)` (lesson B24-L2).
2. Convert to float32 in [0, 1]: `.float() / 255`. In torchvision, `v2.ToImage()` then `v2.ToDtype(torch.float32, scale=True)` do steps 1 and 2 on an `(H, W, C)` uint8 array.
3. Normalise each channel with the training images' mean and standard deviation: `v2.Normalize(mean, std)` computes `(x - mean) / std` per channel. A pretrained model needs the statistics it was trained with (lesson B24-L5).

Augmentation makes small random changes to each training image every time it is loaded: a flip, a shift of a few pixels, a small rotation, a change of brightness. The model meets new variants every epoch, so it learns the object rather than the exact pixels.

```
train_tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
    v2.RandomHorizontalFlip(),
                       v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(MEAN, STD)])
eval_tf = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
    scale=True), v2.Normalize(MEAN, STD)])
```

`v2.RandomCrop(32, padding=4)` pads by 4 pixels and cuts a random 32 by 32 window: a shift of up to 4 pixels. `v2.RandomHorizontalFlip()` flips half the time (`p=0.5`).

Three rules:

- **Augment training images only.** Validation and test images get the fixed steps alone. With random changes, the same validation set gave different inputs on two calls, so its metric would change between runs.

- **Keep the label true.** A flipped photo of a shop still shows a shop. A flipped handwritten 2 is no longer a 2, and a 6 rotated half a turn reads as 9.
- **Transform in `__getitem__`**, so every epoch draws fresh changes.

Why it matters

A model of shelf photos scored 0.91 accuracy on validation one day and 0.88 the next, with no code change. The validation `Dataset` had been built with the training transform, so its images were randomly cropped and flipped on every evaluation. With the fixed transform, validation accuracy was the same on every run, and early stopping chose its best epoch on a stable number.

Worked example: weak code

```
tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
  v2.RandomVerticalFlip(), v2.ToDtype(torch.float32)])
train_rows = ImageRows(train_images, train_labels, transform=tf)
val_rows = ImageRows(val_images, val_labels, transform=tf)
```

1. **Line 1.** No scaling to [0, 1], no normalisation, and an upside-down flip that changes what many images show.
2. **Line 3.** Random changes on validation: the metric moves between runs.

Worked example: strong code

```
import numpy as np
import torch
from torch.utils.data import Dataset
from torchvision.transforms import v2


def build_transforms(mean: list[float], std: list[float]) ->
  tuple[v2.Compose, v2.Compose]:
    """Training transform with augmentation, and the fixed transform for
  evaluation.

    Args:
        mean: Per-channel mean of the training images, in [0, 1].
        std: Per-channel standard deviation of the training images.

    Returns:
        (train_transform, eval_transform).
    """
    fixed = [v2.ToDtype(torch.float32, scale=True), v2.Normalize(mean,
  std)]
    train_tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
  v2.RandomHorizontalFlip(), *fixed])
    eval_tf = v2.Compose([v2.ToImage(), *fixed])
```

```
    return train_tf, eval_tf


class ImageRows(Dataset[tuple[torch.Tensor, int]]):
    """(H, W, C) uint8 images with integer labels, transformed on every access."""

    def __init__(self, images: np.ndarray, labels: np.ndarray, transform: v2.Compose) -> None:
        self.images, self.labels, self.transform = images, labels, transform

    def __len__(self) -> int:
        return len(self.labels)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, int]:
        return self.transform(self.images[i]), int(self.labels[i])
```

1. **Line 17.** Scaling and normalisation, shared by both transforms so they cannot drift apart.
2. **Line 18.** Small shifts and left-right flips: changes that keep a photo's label.
3. **Line 19.** No randomness for validation and test.
4. **Line 33.** Transform on access, so each epoch sees new variants.

Channels first, float32 in [0, 1], normalised with training statistics. Augment training images only, with changes that keep the label true.

Recap card

- **Image to model input** (H, W, C) uint8 to (C, H, W) float32 in [0, 1], then normalise per channel with training statistics.
- **torchvision v2 steps for a uint8 image** v2.ToImage(), v2.ToDtype(torch.float32, scale=True), v2.Normalize(mean, std).
- **Where augmentation goes** Training transform only, applied in **getitem** so each epoch draws new changes. Validation and test get the fixed steps.
- **Choosing augmentations** Only changes that keep the label true: shifts and left-right flips for photos, never flips or half-turns for digits.

Practice

Exercise 1. Multiple choice (level 1)

Should the validation `Dataset` use the training transform, with `RandomCrop` and `RandomHorizontalFlip`?

- a) No. Validation gets the fixed steps only, so its metric measures the same images on every run

- b) Yes. Validation should see the same variety as training
- c) Yes, but only for the final test set
- d) No, because augmentation slows evaluation down

Why?

- r1) Augmentation adds variety for learning. A metric needs fixed inputs, or it changes between runs for no reason.
- r2) Augmentation leaks the labels.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import numpy as np
import torch

img = np.zeros((32, 48, 3), dtype=np.uint8)
t = torch.from_numpy(img).permute(2, 0, 1)
print(tuple(t.shape), t.dtype)
```

Exercise 3. Fill the gap (level 2)

Complete the fixed evaluation steps: float32 in [0, 1], then per-channel normalisation.

```
eval_tf = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, _____(1)=True),
    v2._____(2)(MEAN, STD),
])
```

Exercise 4. Write the function (level 3)

Write `channel_stats(images) -> tuple[np.ndarray, np.ndarray]`: the per-channel mean and standard deviation of training images, for `v2.Normalize`.

- `images` is a uint8 array of shape `(N, H, W, C)`. Scale to [0, 1] by dividing by 255 first.
- Return two float arrays of length `C`, taken over every image, row and column (`axis=(0, 1, 2)`).
- Raise `ValueError` when the array is not 4-D or holds no images.

Your code must pass these checks:

- Average over images, rows and columns separately for each channel, after dividing by 255.
- Divide by 255 first, so values lie in [0, 1].
- Raise ValueError for an array that is not 4-D or holds no images.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Per-channel mean and standard deviation of uint8 images, scaled to [0, 1].
def channel_stats(
    images: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]: ...
```

Starter code:

```
import numpy as np


def channel_stats(images):
    return images.mean(), images.std()
```

Exercise 5. Write the function (level 3)

Write `random_shift(image, max_shift, rng) -> np.ndarray`, a NumPy version of `v2.RandomCrop(size, padding=max_shift)` for a 2-D image.

- Pad the image with `max_shift` zeros on every side, using `np.pad`.
- Draw `top, left = rng.integers(0, 2 * max_shift + 1, size=2)`, once per call.
- Return the window `padded[top : top + h, left : left + w]`, the original size.
- Raise `ValueError` when `image` is not 2-D or `max_shift` is negative.

What you need to know.

`np.pad(a, n)` adds `n` zeros on every side of every dimension:

```
np.pad(np.array([[5]]), 1)    # array([[0, 0, 0], [0, 5, 0], [0, 0, 0]])
```

`rng.integers(low, high, size=2)` draws two whole numbers from `low` up to, but not including, `high`.

Your code must pass these checks:

- Pad by max_shift, draw top and left with one rng.integers(0, 2 * max_shift + 1, size=2), and cut the original size.
- max_shift 0 returns the image unchanged.
- Raise ValueError for a non-2-D image or a negative max_shift.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Shift an image by up to max_shift pixels, filling with zeros.
def random_shift(
    image: np.ndarray,
    max_shift: int,
```

```
    rng: np.random.Generator,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def random_shift(image, max_shift, rng):
    return image
```

Exercise 6. Spot the bug (level 4)

Validation accuracy differs between two evaluations of the same saved model. Tap the line responsible, then fix it.

```
import torch
from torchvision.transforms import v2

train_tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
    v2.ToDtype(torch.float32, scale=True)])
eval_tf = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
    scale=True)])

train_rows = ImageRows(train_images, train_labels, transform=train_tf)
val_rows = ImageRows(val_images, val_labels, transform=train_tf)
```

Write down the line number of the bug. Then write the fix.

Sources

- torchvision 0.29 with PyTorch 2.15: v2.ToImage then v2.ToDtype(torch.float32, scale=True) turns an (H, W, C) uint8 array into a (C, H, W) float32 image in [0, 1]; the training pipeline with RandomCrop(32, padding=4) and RandomHorizontalFlip gives different outputs on two calls, the evaluation pipeline the same; RandomHorizontalFlip defaults to p=0.5; permute(2, 0, 1) keeps uint8. (verified: Run on PyTorch 2.15.0 and torchvision 0.29.2 (PyPI wheels, CPU), November 2026.)

B24-L5 Transfer learning, a pretrained backbone with a new head

Concept

A CNN trained from scratch needs many labelled images. **Transfer learning** starts from a network already trained on a large dataset, such as ImageNet's

1,000 classes, and reuses its filters, which detect edges, textures and shapes that most photos share (torchvision 0.29, PyTorch 2.15):

```
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes)
optimizer = torch.optim.AdamW((p for p in model.parameters() if
   p.requires_grad), lr=1e-3)
preprocess = weights.transforms()
```

1. **Load** the weights. `ResNet18_Weights.DEFAULT` is `IMAGENET1K_V1`. They download from download.pytorch.org on first use.
2. **Freeze** the backbone. With `requires_grad = False`, autograd computes no gradients for those weights (lesson B23-L2).
3. **Replace the head.** ResNet-18 ends in `model.fc`, a `Linear(512, 1000)`. A new `Linear(512, n_classes)` trains from scratch: for 3 classes, 1,539 of 11.7 million parameters.
4. **Build the optimiser after the new head**, from the trainable parameters. An optimiser built earlier never sees the head, and the head never changes.
5. **Preprocess as the backbone was trained.** `weights.transforms()` resizes to 256, crops the centre 224 and normalises with ImageNet's mean `[0.485, 0.456, 0.406]` and standard deviation `[0.229, 0.224, 0.225]`.

Batch norm still updates in training mode. Freezing stops gradient updates, but in `model.train()` the batch-norm layers keep updating their running mean and variance from your batches. In a check, a frozen ResNet-18's first batch-norm layer changed its running mean after one training-mode pass, and kept it in eval mode. Put the frozen backbone in eval mode while the head trains.

Fine-tuning comes next: unfreeze the last block, `model.layer4`, and train it with a smaller learning rate than the head, through optimiser parameter groups: `AdamW([{"params": model.layer4.parameters(), "lr": 1e-4}, {"params": model.fc.parameters(), "lr": 1e-3}])`.

Why it matters

A team had 400 labelled photos of store displays in four layouts. A CNN trained from scratch reached 0.55 validation accuracy and overfitted within ten epochs. A frozen pretrained ResNet-18 with a new four-class head, fed

with the weights' own preprocessing, gave a model the team could use after one afternoon. They wrote down that the backbone's weights came from ImageNet, so the model card named its training data.

Worked example: weak code

```
model = resnet18(weights=ResNet18_Weights.DEFAULT)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
model.fc = nn.Linear(512, 4)
tf = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
  v2.Normalize(OUR_MEAN, OUR_STD)])
```

1. **Line 2.** Built before the new head and with nothing frozen: it updates the old backbone and never the new head.
2. **Line 4.** Statistics the backbone never saw, and no resize to the 224 crop it was trained on.

Worked example: strong code

```
import torch
from torch import nn
from torchvision.models import ResNet18_Weights, resnet18


def build_transfer_model(n_classes: int) -> tuple[nn.Module,
  torch.optim.Optimizer, object]:
    """Frozen pretrained ResNet-18 with a new head, its optimiser and its
  preprocessing.

    Args:
        n_classes: Number of classes in your data.

    Returns:
        The model, an AdamW optimiser over the head only, and the weights'
  transforms.
    """
    weights = ResNet18_Weights.DEFAULT
    model = resnet18(weights=weights)
    for param in model.parameters():
        param.requires_grad = False
    model.fc = nn.Linear(model.fc.in_features, n_classes)
    optimizer = torch.optim.AdamW((p for p in model.parameters() if
  p.requires_grad), lr=1e-3)
    return model, optimizer, weights.transforms()


def set_training_mode(model: nn.Module) -> None:
    """Train the head while the frozen backbone keeps its batch-norm
  statistics."""
    model.eval()
```

```
27      model.fc.train()
```

1. **Line 18.** No gradients, so no updates, for the pretrained filters.
2. **Line 19.** A new head sized from the old one's input: 512 for ResNet-18.
3. **Line 20.** Built after the new head, from trainable parameters only.
4. **Line 21.** The resize, crop and ImageNet normalisation the backbone expects.
5. **Line 26.** Batch-norm layers stop updating their running statistics. The head has none, so train mode changes nothing for it.

Load pretrained weights, freeze them, replace the head, build the optimiser afterwards from trainable parameters, and preprocess exactly as the backbone was trained.

Recap card

- **Transfer learning recipe** Load pretrained weights, freeze them, replace the head, build the optimiser afterwards from trainable parameters.
- **ResNet-18 head** model.fc is Linear(512, 1000). Replace it with nn.Linear(model.fc.in_features, n_classes).
- **Preprocessing for a pretrained model** weights.transforms(): resize 256, centre crop 224, ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225].
- **Frozen batch norm in train mode** Its running statistics still update. Keep the frozen backbone in eval mode while the head trains.

Practice

Exercise 1. Multiple choice (level 1)

You have 300 labelled photos in 3 classes. Which plan is most likely to work?

- a) A pretrained ResNet-18, frozen, with a new 3-class head and the weights' own preprocessing
- b) A new ResNet-18 trained from scratch on the 300 photos
- c) A pretrained ResNet-18 with all layers trained at a high learning rate
- d) A pretrained ResNet-18 with its 1,000-class head, mapping its predictions to your classes

Why?

- r1) The pretrained filters already detect general image features, so 300 photos only need to train a small head.
- r2) Frozen models run faster on a CPU.

Exercise 2. Predict the output (level 2)

What does this print? `weights=None` builds the architecture without downloading anything. (torchvision 0.29)

What you need to know.

`model.named_parameters()` yields `(name, parameter)` pairs. Names follow the attribute path, so a layer stored as `model.fc` gives `fc.weight` and `fc.bias`.

```
from torch import nn
from torchvision.models import resnet18

model = resnet18(weights=None)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 3)
trainable = [name for name, p in model.named_parameters() if
  p.requires_grad]
print(trainable, sum(p.numel() for p in model.parameters() if
  p.requires_grad))
```

Exercise 3. Fill the gap (level 2)

Freeze the backbone, and size the new head from the old one.

```
for param in model.parameters():
    param._____(1) = False
model.fc = nn.Linear(model.fc._____(2), 3)
```

Exercise 4. Write the function (level 3)

Write `normalise_chw(image, mean, std) -> np.ndarray`: normalise a `(C, H, W)` float image in [0, 1] channel by channel, as `v2.Normalize` does with a pretrained model's statistics.

- Return `(image - mean[c]) / std[c]` for each channel `c`.
- Raise `ValueError` when the image is not 3-D, when `mean` or `std` has a length other than `C`, or when any `std` is not positive.

What you need to know.

`arr[:, None, None]` adds two dimensions of size 1, so a length-C array broadcasts across a `(C, H, W)` image one channel at a time:

```
np.array([1.0, 2.0])[:, None, None].shape    # (2, 1, 1)
```

Your code must pass these checks:

- Subtract each channel's mean and divide by its standard deviation.
- Match statistics to channels, not to columns: mean[c] for channel c.
- Raise ValueError for a non-3-D image, mismatched lengths, or a non-positive std.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Normalise a (C, H, W) image channel by channel.
def normalise_chw(
    image: np.ndarray,
    mean: np.ndarray,
    std: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def normalise_chw(image, mean, std):
    return (image - mean) / std
```

Exercise 5. Architecture design (level 3)

Plan a classifier for 2,000 labelled shelf photos in 5 classes, starting from a pretrained ResNet-18.

Options: Freeze the backbone and train a new 5-class head, Train every layer from the start at lr 1e-3, Start from random weights.

1. First phase
2. Preprocessing
3. When the head has converged

Exercise 6. Spot the bug (level 4)

Training loss is identical in every epoch, and the head's weights never change. Tap the line responsible, then fix it.

```python
import torch
from torch import nn
from torchvision.models import ResNet18_Weights, resnet18

model = resnet18(weights=ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
model.fc = nn.Linear(model.fc.in_features, 4)
```

Write down the line number of the bug. Then write the fix.

Sources

- torchvision 0.29: resnet18(weights=None) has 11,689,512 parameters and fc = Linear(512, 1000); after freezing, a new Linear(512, 3) head gives 1,539 trainable parameters named fc.weight and fc.bias;

ResNet18_Weights.DEFAULT is IMAGENET1K_V1 and its transforms() resize to 256, crop 224 and normalise with mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]; frozen batch norm updates running statistics in train mode and not in eval mode; an optimiser built before the head is replaced leaves the head unchanged; parameter groups take their own lr. (verified: Run on PyTorch 2.15.0 and torchvision 0.29.2 (PyPI wheels, CPU) with weights=None, November 2026.)

- resnet18(weights=ResNet18_Weights.DEFAULT) downloads the ImageNet weights from download.pytorch.org on first use; that the pretrained weights load and classify as documented. (unverified: Weights URL read from ResNet18_Weights.DEFAULT.url. download.pytorch.org was blocked in the build session, so the weights were not loaded.)

B24-L6 Evaluating image classifiers, per class and on honest splits

Concept

A single accuracy figure hides which classes fail. Evaluate an image classifier in four steps.

1. Predict in eval mode. Call `model.eval()`, wrap the loop in `torch.no_grad()` (lesson B16-L2), and take the class with the largest logit: `preds = logits.argmax(dim=1)` (PyTorch 2.15).

2. Build a confusion matrix. `sklearn.metrics.confusion_matrix(y_true, y_pred)` (scikit-learn 1.8) counts true classes in rows and predicted classes in columns. Each row's diagonal entry divided by the row total is that class's **recall**:

```
cm = confusion_matrix([0, 0, 1, 1, 2, 2], [0, 1, 1, 1, 2, 0])
# [[1, 1, 0], [0, 2, 0], [1, 0, 1]]
recall = cm.diagonal() / cm.sum(axis=1)     # [0.5, 1.0, 0.5]
```

Guard the division: a class with no true images has a row total of 0. Large off-diagonal cells name the pairs the model confuses.

3. Look at the mistakes. Show the misclassified images with their true and predicted labels, the most confident errors first. Many turn out to be wrong labels, blurred photos or two classes that people also mix up.

4. Check the split is honest. Photos taken in bursts, the same shelf on two days, or augmented copies saved as files are near-duplicates. If one lands in training and its twin in validation, the model is graded on images it has

nearly seen. Split by the unit that produces near-duplicates, such as the shelf, store or session (lesson B11-L1). `np.unique(groups)` lists the distinct groups, so you can shuffle groups, not photos. Then check data from a new source, such as another camera or season, apart from the rest.

Report accuracy with an interval (lesson B8-L2), and recall for every class, beside the confusion matrix.

Why it matters

A shelf classifier reported 0.97 validation accuracy. Each shelf had been photographed in bursts of five, and the split was by photo, so nearly every validation image had a twin in training. Split by shelf, accuracy fell to 0.84, and the confusion matrix showed one class, partly empty shelves, at a recall of 0.52. That number, not 0.97, decided what the team fixed next.

Worked example: weak code

```
order = rng.permutation(len(photos))
val, train = order[:400], order[400:]
accuracy = (model(X[val]).argmax(dim=1) == y[val]).float().mean()
print(f"accuracy: {accuracy:.2f}")
```

1. **Line 1.** Split by photo: burst twins land on both sides.
2. **Line 3.** Train mode, gradients tracked, and one number that hides every class.

Worked example: strong code

```
import numpy as np
import torch
from sklearn.metrics import confusion_matrix
from torch import nn


def split_by_group(groups: np.ndarray, val_share: float, rng:
    np.random.Generator) -> tuple[np.ndarray, np.ndarray]:
    """Training and validation row positions, with no group on both
    sides."""
    shelves = rng.permutation(np.unique(groups))
    n_val = round(len(shelves) * val_share)
    in_val = np.isin(groups, shelves[:n_val])
    return np.flatnonzero(~in_val), np.flatnonzero(in_val)


def predict(model: nn.Module, X: torch.Tensor) -> np.ndarray:
    """Class predictions in eval mode, without gradients."""
    model.eval()
    with torch.no_grad():
        return model(X).argmax(dim=1).numpy()

```

```
21
22 def per_class_recall(y_true: np.ndarray, y_pred: np.ndarray, n_classes:
     int) -> np.ndarray:
23     """Recall for each class. NaN where a class has no true examples."""
24     cm = confusion_matrix(y_true, y_pred, labels=list(range(n_classes)))
25     totals = cm.sum(axis=1)
5      return np.where(totals > 0, cm.diagonal() / np.maximum(totals, 1),
     np.nan)
```

1. **Line 9.** Shuffle shelves, not photos, so twins stay together.
2. **Line 12.** Row positions for each side. No shelf appears in both.
3. **Line 17.** Dropout off and batch-norm statistics fixed.
4. **Line 19.** The class with the largest logit.
5. **Line 26.** No division by zero, and a missing class shows as NaN, not as 0 or 1.

Predict in eval mode, read recall per class from the confusion matrix, look at the errors, and split by the unit that makes near-duplicates.

Recap card

- **Class predictions from a classifier** model.eval(), torch.no_grad(), then logits.argmax(dim=1).
- **Recall per class from a confusion matrix** Diagonal over row totals (rows are true classes). Guard empty rows: NaN, not 0.
- **Near-duplicate images** Bursts, repeat shots and saved augmentations. Split by shelf, store or session, never by photo.
- **What to report for an image classifier** Accuracy with an interval, recall for every class, the confusion matrix, and a look at the worst errors.

Practice

Exercise 1. Multiple choice (level 1)

A classifier reaches 0.95 accuracy. One of its five classes makes up 5% of the images. What do you check before trusting the result?

- a) The confusion matrix and the recall of each class
- b) Nothing more: 0.95 is high
- c) The training loss curve
- d) Accuracy on the training set

Why?

- r1) A model can miss the small class entirely and still score 0.95. Recall per class shows it.
- r2) Confusion matrices are the standard in image papers.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix([0, 0, 1, 1, 2, 2], [0, 1, 1, 1, 2, 0])
print(cm.tolist(), (cm.diagonal() / cm.sum(axis=1)).tolist())
```

Exercise 3. Fill the gap (level 2)

Predict classes in eval mode without tracking gradients.

```
model.eval()
with torch._____(1)():
    preds = model(X)._____(2)(dim=1)
```

Exercise 4. Write the function (level 3)

Write `confusion(y_true, y_pred, n_classes) -> np.ndarray` without scikit-learn: an `n_classes` by `n_classes` integer array with true classes in rows and predicted classes in columns.

- Raise `ValueError` when the arrays differ in length or hold a label outside `0` to `n_classes - 1`.

What you need to know.

`np.concatenate([a, b])` joins arrays end to end, so one `min()` and `max()` cover both:

```
np.concatenate([np.array([3, 1]), np.array([7])])    # array([3, 1, 7])
```

Your code must pass these checks:

- Count each (true, predicted) pair, true classes in rows.
- Size the matrix from n_classes, with integer counts, even for unused classes.
- Raise ValueError for different lengths or labels outside 0 to n_classes - 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Confusion matrix: true classes in rows, predicted classes in columns.
def confusion(
    y_true: np.ndarray,
    y_pred: np.ndarray,
    n_classes: int,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np

```

```
3
4 def confusion(y_true, y_pred, n_classes):
5     return np.zeros((n_classes, n_classes), dtype=int)
```

Exercise 5. Write the function (level 3)

Write `recall_by_class(cm) -> np.ndarray` from a square confusion matrix (true classes in rows).

- Return the diagonal divided by the row totals, as floats.
- Return `np.nan` for a class whose row total is 0, without a division warning.
- Raise `ValueError` when `cm` is not square.

Your code must pass these checks:

- Divide the diagonal by the row totals.
- Return NaN for a class with no true examples, with no division warning.
- Raise ValueError when the matrix is not square.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Recall per class from a confusion matrix with true classes in rows.
2 def recall_by_class(cm: np.ndarray) -> np.ndarray: ...
```

Starter code:

```
1 import numpy as np
2
3
4 def recall_by_class(cm):
5     return cm.diagonal() / cm.sum(axis=1)
```

Exercise 6. Spot the bug (level 4)

Every shelf was photographed in a burst of five. Validation accuracy is far higher than on photos of new shelves. Tap the line responsible, then fix it.

What you need to know.

`np.unique(a)` returns the distinct values of `a`, sorted. `np.flatnonzero(mask)` returns the positions where a boolean array is `True`:

```
1 np.unique(np.array(["b", "a", "b"]))              # array(['a', 'b'])
2 np.flatnonzero(np.array([False, True, True]))     # array([1, 2])
```

```
1 import numpy as np
2
3
4 def split_photos(shelf_ids, val_share, rng):
5     order = rng.permutation(len(shelf_ids))
6     n_val = round(len(shelf_ids) * val_share)
7     return order[n_val:], order[:n_val]
```

Write down the line number of the bug. Then write the fix.

Practical: Train a CNN on handwritten digits and compare it with a dense network

Platform: Laptop. Work in `labs/cnn/` in this repository. It uses scikit-learn's bundled 8 by 8 handwritten digits, so nothing downloads. You write the CNN steps from module B24: the channel dimension, a shift augmentation, the model, a confusion matrix and per-class recall. The training script trains your CNN and a dense network on the same split, with early stopping. You need PyTorch 2.15 on the laptop. When the CNN beats the dense network, push the lab to your own GitHub repository: it is the roadmap's CNN evidence for months 4 to 6.

1. Run `cd labs/cnn`, create a virtual environment, then `pip install -e .` and `pip install pytest`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/digits_cnn/`.
2. In `src/digits_cnn/vision.py`, write `add_channel` (lessons B24-L2 and B24-L4) and `random_shift` (lesson B24-L4). Run `pytest tests/test_vision.py -k "channel or shift"`.
3. Write `confusion` and `recall_by_class` in the same file (lesson B24-L6).
4. In `src/digits_cnn/models.py`, write `make_cnn` (lessons B24-L1 to B24-L3). Run `pytest`: every test passes.
5. Run `python -m digits_cnn train`. It prints validation accuracy for the dense network and the CNN, and the CNN's recall for each digit. Which digits does it confuse?
6. Run `python -m digits_cnn train --augment`, which shifts training images by up to one pixel. Compare both models with and without it, and explain the result (lesson B24-L4).
7. Write your README: the data, both models, their parameter counts, the comparison, the confusion matrix, and what augmentation changed. Push the folder to a new GitHub repository.

Sources

- scikit-learn 1.8: confusion_matrix([0, 0, 1, 1, 2, 2], [0, 1, 1, 1, 2, 0]) is [[1, 1, 0], [0, 2, 0], [1, 0, 1]] with rows as true classes, and recall per class [0.5, 1.0, 0.5]; PyTorch 2.15 argmax(dim=1) returns class indices. (verified: Run on scikit-learn 1.8.0 (Pyodide pins) and 1.9.1, PyTorch 2.15.0, November 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B24-L1 Convolution, a small filter slid across an image

B24-L1 exercise 1

Hints. (1) What does each output value look at? (2) Where are the same weights used? (3) Local and shared.

Answer. a) Each output looks at a small neighbourhood, and the same weights are reused at every position Because: r1) Nearby pixels form patterns, and a pattern means the same wherever it appears. Local, shared weights encode both, with far fewer parameters.

A convolution looks at small neighbourhoods and reuses one set of weights at every position, so a pattern learned once is found anywhere.

Common wrong answers.

- `b`: The opposite: weight sharing gives far fewer parameters. 448 against over 44 million in the lesson's example.
- `c`: nn.Conv2d does not flip the kernel, and convolution is not rotation invariant.
- `d`: The kernel weights are learned like any other weights.
- `a|r2`: Right answer, wrong reason. It is about locality and shared weights.

B24-L1 exercise 2

Hints. (1) How many positions does a 2 by 2 kernel have on a 4 by 4 image? (2) Where does brightness change from 0 to 1? (3) At the edge: (-1)(0) + (1)(1), twice.

Answer.

```
1 [[0.0, 2.0, 0.0], [0.0, 2.0, 0.0], [0.0, 2.0, 0.0]]
```

The kernel fits at 3 by 3 positions. Only the middle column straddles the 0-to-1 boundary, giving 2 on every row.

Common wrong answers.

- `[[0.0, 2.0, 0.0, 0.0], [0.0, 2.0, 0.0, 0.0], [0.0, 2.0, 0.0, 0.0], [0.0, 2.0, 0.0, 0.0]]`: Without padding, a 2 by 2 kernel fits 3 times across 4 columns and 3 times down 4 rows: a 3 by 3 map.

- `[[0.0, 1.0, 0.0], [0.0, 1.0, 0.0], [0.0, 1.0, 0.0]]`: The kernel covers two rows. Each row adds -0 + 1 at the edge, so the sum is 2.

B24-L1 exercise 3

Hints. (1) The patch has the kernel's height. (2) Products are added together. (3) `kh` and `sum`.

Answer.

```
out[i, j] = (image[i:i + kh, j:j + kw] * kernel).sum()
```

Take the patch the size of the kernel, multiply element by element, and sum.

Common wrong answers.

- g1 = `kh`; g2 = `mean`: A convolution sums the products. A mean divides by the kernel size.

B24-L1 exercise 4

Hints. (1) Check `ndim` on both inputs, then compute the output size from `kernel.shape`. (2) Raise when either output size is below 1. (3) Two loops, each cell `(image[i : i + kh, j : j + kw] * kernel).sum()`.

Answer.

```
import numpy as np


def correlate2d(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Slide a kernel over an image without padding, as nn.Conv2d does.

    Args:
        image: 2-D array, shape (h, w).
        kernel: 2-D array, shape (kh, kw), no larger than the image.

    Returns:
        The feature map, shape (h - kh + 1, w - kw + 1).

    Raises:
        ValueError: If either input is not 2-D or the kernel is larger than the image.
    """
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError(f"need 2-D arrays, got {image.shape} and {kernel.shape}")
    kh, kw = kernel.shape
    out_h, out_w = image.shape[0] - kh + 1, image.shape[1] - kw + 1
    if out_h < 1 or out_w < 1:
        raise ValueError(f"kernel {kernel.shape} is larger than image {image.shape}")
    out = np.zeros((out_h, out_w))
```

```
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = (image[i : i + kh, j : j + kw] * kernel).sum()
    return out
```

The output size comes from the kernel, the guard stops impossible inputs, and the patch-times-kernel sum without a flip matches `nn.Conv2d`.

B24-L1 exercise 5

Hints. (1) One kernel grid per input channel, per filter. (2) Plus one bias per filter. (3) `out_channels * in_channels * kernel_size ** 2 + out_channels`.

Answer.

```
def conv_params(in_channels: int, out_channels: int, kernel_size: int) ->
  int:
    """Parameters in a square Conv2d layer with bias.

    Args:
        in_channels: Channels in the input.
        out_channels: Filters, one feature map each.
        kernel_size: Height and width of each kernel.

    Returns:
        Weights plus biases.

    Raises:
        ValueError: If any argument is below 1.
    """
    if min(in_channels, out_channels, kernel_size) < 1:
        raise ValueError("every size must be at least 1")
    return out_channels * in_channels * kernel_size**2 + out_channels
```

Each filter spans every input channel, so 16 filters of 3 by 3 over 3 channels hold 16 × 27 weights plus 16 biases: 448. The count does not depend on the image size.

B24-L1 exercise 6

Hints. (1) How many positions does a 2 by 2 kernel have on a 3 by 3 image? (2) Count: rows 0 and 1, so 2 = 3 - 2 + 1. (3) Add 1 to both sizes.

Answer. The bug is on line 6.

Fixed version:

```
import numpy as np


def correlate2d(image, kernel):
    kh, kw = kernel.shape
    out_h, out_w = image.shape[0] - kh + 1, image.shape[1] - kw + 1
    out = np.zeros((out_h, out_w))
```

```
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = (image[i : i + kh, j : j + kw] * kernel).sum()
    return out
```

A kernel of height kh fits at rows 0 to h - kh, which is h - kh + 1 positions. Without the + 1, the last row and column of positions are never computed.

Common wrong answers.

- Line 10: The patch and sum are right. Fewer positions are visited than exist.
- Line 8: The loop follows out_h. The problem is how out_h was computed.

B24-L2 Padding, stride, channels and the (N, C, H, W) layout

B24-L2 exercise 1

Hints. (1) Use (H + 2p - k) // s + 1. (2) Which option has stride 1 and p = (k - 1) / 2? (3) Kernel 3, padding 1.

Answer. a) `nn.Conv2d(3, 16, kernel_size=3, padding=1)` Because: r1) (32 + 2 - 3) // 1 + 1 = 32: one pixel of padding on each side makes up for the kernel's overhang.

With stride 1 and padding `(k - 1) / 2`, the output size equals the input size: 3 and 1 here.

Common wrong answers.

- `b`: Without padding, a 3 by 3 kernel gives 30.
- `c`: Stride 2 halves the size: 16.
- `d`: A 5 by 5 kernel needs padding 2 to keep the size. With padding 1 it gives 30.
- `a|r2`: Right layer, wrong reason. The padding must match the kernel: (k - 1) / 2 for an odd k.

B24-L2 exercise 2

Hints. (1) Channels become out_channels. (2) Apply (H + 2p - k) // s + 1 to each. (3) 30, then 16, then 14.

Answer.

```
(8, 16, 30, 30)
(8, 16, 16, 16)
(8, 16, 14, 14)
```

32 - 3 + 1 = 30. (32 + 2 - 3) // 2 + 1 = 16. (32 - 5) // 2 + 1 = 14. The channels become 16 each time.

Common wrong answers.

- `(8, 3, 30, 30) (8, 3, 16, 16) (8, 3, 14, 14)`: The channel dimension becomes out_channels: 16.
- `(8, 16, 30, 30) (8, 16, 15, 15) (8, 16, 14, 14)`: (32 + 2 - 3) // 2 + 1 = 31 // 2 + 1 = 16. Padding counts on both sides.

B24-L2 exercise 3

Hints. (1) What divides the distance the kernel can travel? (2) Which method reorders dimensions? (3) `stride` and `permute`.

Answer.

```
out = (size + 2 * padding - kernel_size) // stride + 1
batch = images.permute(0, 3, 1, 2)
```

The kernel moves `stride` pixels per step. `permute(0, 3, 1, 2)` moves the channel dimension to position 1. `reshape` to the same shape would mix pixels from different channels.

Common wrong answers.

- g1 = `stride`; g2 = `reshape`: reshape keeps the memory order and scrambles pixels across channels. permute moves dimensions.

B24-L2 exercise 4

Hints. (1) Validate the arguments first. (2) Padding is added on both sides: `2 * padding`. (3) Use `//` and check the result is at least 1.

Answer.

```
def conv_output_size(size: int, kernel_size: int, stride: int = 1, padding: int = 0) -> int:
    """Height or width after one Conv2d or pooling layer.

    Args:
        size: Input height or width.
        kernel_size: Kernel height or width.
        stride: Step between positions.
        padding: Zeros added on each side.

    Returns:
        The output height or width.

    Raises:
        ValueError: If an argument is out of range or the kernel does not fit.
    """
```

```
    if min(size, kernel_size, stride) < 1 or padding < 0:
        raise ValueError(f"invalid sizes: {size}, {kernel_size},
{stride}, {padding}")
    out = (size + 2 * padding - kernel_size) // stride + 1
    if out < 1:
        raise ValueError(f"a {kernel_size} kernel does not fit a padded
size of {size + 2 * padding}")
    return out
```

Padding counts twice, floor division drops a partial last step, and the guards catch layer settings that cannot work before a model is built.

B24-L2 exercise 5

Hints. (1) Loop over the layers, unpacking each tuple. (2) Feed each layer the previous layer's output size. (3) Do not catch the ValueError.

Answer.

```
def stack_output_size(size: int, layers: list[tuple[int, int, int]]) ->
int:
    """Height or width after a stack of conv or pooling layers.

    Args:
        size: Input height or width.
        layers: (kernel_size, stride, padding) for each layer, in order.

    Returns:
        The final height or width.

    Raises:
        ValueError: If a layer does not fit its input.
    """
    for kernel_size, stride, padding in layers:
        size = conv_output_size(size, kernel_size, stride, padding)
    return size
```

Each layer reads the previous layer's output. On 8 by 8 digits, two padded 3 by 3 convolutions with 2 by 2 pooling after each give 8, 4, 4, 2.

B24-L2 exercise 6

Hints. (1) Which dimension does Conv2d read as channels? (2) What layout do the scans arrive in? (3) permute(0, 3, 1, 2).

Answer. The bug is on line 9.

Fixed version:

```
import numpy as np
import torch
from torch import nn

```

```
5  model = nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
       nn.Flatten(), nn.Linear(16 * 28 * 28, 10))
6
7
8  def to_batch(images: np.ndarray) -> torch.Tensor:
9      return torch.as_tensor(images, dtype=torch.float32).permute(0, 3, 1,
       2)
10
11
12 logits = model(to_batch(scans))
```

The scans are (N, H, W, C), so Conv2d reads 28, the height, as the channel count. `permute(0, 3, 1, 2)` moves the single channel to dimension 1.

Common wrong answers.

- Line 5: Conv2d(1, 16, 3, padding=1) is right for single-channel scans. The input layout is wrong.
- Line 12: The call is fine. The batch it receives has channels last.

B24-L3 Pooling, flattening and a complete small CNN

B24-L3 exercise 1

Hints. (1) Does max pooling have parameters? (2) What happens to a feature that moves by one pixel within a block? (3) Smaller maps, shift tolerance.

Answer. a) It halves height and width with no weights, and makes features tolerate small shifts Because: r1) Keeping the largest value in each 2 by 2 block reports that a feature is present nearby, not its exact pixel, and quarters the work for later layers.

Max pooling summarises each block by its largest value: smaller maps, no weights, and some tolerance to small shifts.

Common wrong answers.

- `b`: Max pooling has no weights. It keeps the largest value by rule.
- `c`: Pooling works on each channel separately and keeps the channel count.
- `d`: Pooling comes after the activation. It does not replace it.
- `a|r2`: Right effect, wrong reason. It summarises feature maps, not the raw image.

B24-L3 exercise 2

Hints. (1) Split the 4 by 4 grid into four 2 by 2 blocks. (2) Max of each block, then mean of each block. (3) Top-left block: 1, 3, 4, 2.

Answer.

```
[[4.0, 5.0], [6.0, 3.0]] [[2.5, 2.0], [2.25, 1.75]]
```

The blocks are {1, 3, 4, 2}, {2, 0, 1, 5}, {0, 1, 2, 6} and {3, 2, 1, 1}. Their maxima are 4, 5, 6, 3, and their means 2.5, 2.0, 2.25, 1.75.

Common wrong answers.

- `[[4.0, 3.0, 5.0], [4.0, 3.0, 5.0], [6.0, 6.0, 3.0]] [[2.5, 2.0], [2.25, 1.75]]`: MaxPool2d(2) moves 2 pixels at a time by default, so blocks do not overlap: a 2 by 2 result.
- `[[4.0, 5.0], [6.0, 3.0]] [[4.0, 5.0], [6.0, 3.0]]`: AvgPool2d averages each block: (1 + 3 + 4 + 2) / 4 = 2.5 for the first.

B24-L3 exercise 3

Hints. (1) Which layer halves height and width? (2) Which layer turns (N, C, H, W) into (N, C * H * W)? (3) `MaxPool2d` and `Flatten`.

Answer.

```
nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Flatten(),
nn.Linear(32 * 2 * 2, 10),
```

Pooling halves the map, and `Flatten` lays out the 32 × 2 × 2 values as one row per image for the Linear layer.

Common wrong answers.

- g1 = `MaxPool2d`; g2 = `ReLU`: Linear needs a (N, features) input. Flatten turns (N, C, H, W) into that.

B24-L3 exercise 4

Hints. (1) Output size: `x.shape[0] // size` by `x.shape[1] // size`. (2) Block (i, j) starts at row `i * size` and column `j * size`. (3) Take `.max()` of each block.

Answer.

```
import numpy as np


def max_pool2d(x: np.ndarray, size: int = 2) -> np.ndarray:
    """Max pooling with non-overlapping size by size blocks.

```

```
    Args:
        x: 2-D feature map.
        size: Block height and width, also the step.

    Returns:
        Array of shape (h // size, w // size).

    Raises:
        ValueError: If x is not 2-D, size is below 1, or x is smaller than one block.
    """
    if x.ndim != 2 or size < 1:
        raise ValueError(f"need a 2-D array and size >= 1, got {x.shape} and {size}")
    out_h, out_w = x.shape[0] // size, x.shape[1] // size
    if out_h < 1 or out_w < 1:
        raise ValueError(f"{x.shape} is smaller than one {size} by {size} block")
    out = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            out[i, j] = x[i * size : (i + 1) * size, j * size : (j + 1) * size].max()
    return out
```

Non-overlapping blocks step by `size`, so the output is `h // size` by `w // size`, and leftover rows and columns are dropped, as in `nn.MaxPool2d`.

B24-L3 exercise 5

Hints. (1) Padded 3 by 3 convs keep the size. Only the pools change it. (2) Halve with `//= 2` once per block, and check it stays at least 1. (3) Return `channels * size * size`.

Answer.

```
def flatten_features(image_size: int, blocks: int, channels: int) -> int:
    """Inputs to the Linear layer after conv, ReLU, MaxPool2d(2) blocks.

    Args:
        image_size: Height and width of the square input.
        blocks: Number of blocks.
        channels: Filters in the last conv layer.

    Returns:
        channels * size * size after the last pool.

    Raises:
        ValueError: If an argument is out of range or the map shrinks below 1.
    """
    if image_size < 1 or channels < 1 or blocks < 0:
```

```
16          raise ValueError(f"invalid arguments: {image_size}, {blocks},
    {channels}")
17      size = image_size
18      for _ in range(blocks):
19          size //= 2
20          if size < 1:
21              raise ValueError(f"{blocks} blocks shrink a
    {image_size} image below 1 pixel")
22      return channels * size * size
```

Pooling rounds down, so 7 becomes 3. The Linear layer needs channels × height × width inputs.

B24-L3 exercise 6

Hints. (1) The error says each image flattens to 128 values. (2) What size are the maps after two MaxPool2d(2) layers on 8 by 8? (3) 32 channels of 2 by 2.

Answer. The bug is on line 7.

Fixed version:

```
1 from torch import nn
2
3 model = nn.Sequential(
4     nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
5     nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
6     nn.Flatten(),
7     nn.Linear(32 * 2 * 2, 10),
8 )
```

Two pools shrink 8 by 8 to 2 by 2, so Flatten gives 32 × 2 × 2 = 128 values. The Linear layer was sized as if no pooling happened.

Common wrong answers.

- Line 6: Flatten is right. It produces 128 values per image, as the error says.
- Line 5: The second block is fine. Its pool halves 4 by 4 to 2 by 2.

B24-L4 Images as tensors, normalisation and augmentation

B24-L4 exercise 1

Hints. (1) What should a metric depend on? (2) Would two evaluations of the same model agree? (3) Fixed steps only.

Answer. a) No. Validation gets the fixed steps only, so its metric measures the same images on every run Because: r1) Augmentation adds variety for

learning. A metric needs fixed inputs, or it changes between runs for no reason.

Augmentation is for learning. Evaluation needs the same inputs every time, so the metric reflects the model alone.

Common wrong answers.

- `b`: Then validation loss changes between calls on the same model, and early stopping chases noise.
- `c`: The test set needs fixed inputs most of all. It is the number you report.
- `d`: Speed is not the problem. A random input makes the metric itself random.
- `a|r2`: Right answer, wrong reason. Augmentation does not leak labels. It makes the metric unstable.

B24-L4 exercise 2

Hints. (1) Old dimensions: 0 is height, 1 is width, 2 is channels. (2) New order: 2, 0, 1. (3) Does permute change the dtype?

Answer.

```
1 (3, 32, 48) torch.uint8
```

`permute(2, 0, 1)` gives (channels, height, width) = (3, 32, 48). The values stay uint8 until `.float() / 255` or `v2.ToDtype(..., scale=True)`.

Common wrong answers.

- `(3, 32, 48) torch.float32`: permute only reorders dimensions. The dtype stays uint8 until you convert it.
- `(3, 48, 32) torch.uint8`: permute(2, 0, 1) puts old dimension 2 first, then 0 (height), then 1 (width).

B24-L4 exercise 3

Hints. (1) Which argument divides uint8 values by 255? (2) Which step computes (x - mean) / std? (3) `scale` and `Normalize`.

Answer.

```
1 eval_tf = v2.Compose([
2     v2.ToImage(),
3     v2.ToDtype(torch.float32, scale=True),
4     v2.Normalize(MEAN, STD),
5 ])
```

`scale=True` maps 0 to 255 onto 0 to 1, and `v2.Normalize` subtracts the mean and divides by the standard deviation for each channel.

Common wrong answers.

- g1 = `scale`; g2 = `RandomCrop`: Random steps do not belong in the evaluation transform. Normalize applies the training statistics.

B24-L4 exercise 4

Hints. (1) Check `images.ndim` and `len(images)` first. (2) Divide a float copy by 255. (3) `.mean(axis=(0, 1, 2))` keeps one value per channel.

Answer.

```python
import numpy as np


def channel_stats(images: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Per-channel mean and standard deviation of uint8 images, scaled to [0, 1].

    Args:
        images: Training images, shape (N, H, W, C), values 0 to 255.

    Returns:
        Mean and standard deviation, each of length C.

    Raises:
        ValueError: If the array is not 4-D or holds no images.
    """
    if images.ndim != 4 or len(images) == 0:
        raise ValueError(f"expected (N, H, W, C) with N >= 1, got {images.shape}")
    scaled = images.astype(np.float64) / 255
    return scaled.mean(axis=(0, 1, 2)), scaled.std(axis=(0, 1, 2))
```

Reducing over images, rows and columns leaves one statistic per channel, on the same [0, 1] scale that `v2.ToDtype(..., scale=True)` produces.

B24-L4 exercise 5

Hints. (1) `padded = np.pad(image, max_shift)`. (2) One draw: `rng.integers(0, 2 * max_shift + 1, size=2)`. (3) Cut `padded[top : top + h, left : left + w]`.

Answer.

```python
import numpy as np


def random_shift(image: np.ndarray, max_shift: int, rng: np.random.Generator) -> np.ndarray:
    """Shift an image by up to max_shift pixels, filling with zeros.

    Args:
        image: 2-D image.
```

```
        max_shift: Largest shift in any direction.
        rng: Seeded generator.

    Returns:
        An array with the image's shape.

    Raises:
        ValueError: If image is not 2-D or max_shift is negative.
    """
    if image.ndim != 2 or max_shift < 0:
        raise ValueError(f"need a 2-D image and max_shift >= 0, got {image.shape}, {max_shift}")
    h, w = image.shape
    padded = np.pad(image, max_shift)
    top, left = rng.integers(0, 2 * max_shift + 1, size=2)
    return padded[top : top + h, left : left + w]
```

Padding by `max_shift` and cutting a window anywhere from offset 0 to `2 * max_shift` shifts the content by up to `max_shift` pixels in either direction, with zeros filling the gap.

B24-L4 exercise 6

Hints. (1) Which transforms are random? (2) Which Dataset should never see them? (3) transform=eval_tf for validation.

Answer. The bug is on line 8.

Fixed version:

```
1 import torch
2 from torchvision.transforms import v2
3
4 train_tf = v2.Compose([v2.ToImage(), v2.RandomCrop(32, padding=4),
    v2.ToDtype(torch.float32, scale=True)])
5 eval_tf = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
    scale=True)])
6
7 train_rows = ImageRows(train_images, train_labels, transform=train_tf)
8 val_rows = ImageRows(val_images, val_labels, transform=eval_tf)
```

The validation rows use the random training transform, so each evaluation crops different windows. Validation uses `eval_tf`, which has no random steps.

Common wrong answers.

- Line 4: Random cropping is right for training. The problem is where else this transform is used.
- Line 7: Training rows should be augmented. Look at the validation rows.

B24-L5 Transfer learning, a pretrained backbone with a new head

B24-L5 exercise 1

Hints. (1) How many parameters does ResNet-18 have? (2) What do its early layers detect? (3) Reuse the backbone, train a head.

Answer. a) A pretrained ResNet-18, frozen, with a new 3-class head and the weights' own preprocessing Because: r1) The pretrained filters already detect general image features, so 300 photos only need to train a small head.

With little data, reuse a backbone's general features and train only a small new head on top of them.

Common wrong answers.

- `b`: 11.7 million parameters from 300 photos will overfit. Reuse filters learned on a large dataset.
- `c`: A high learning rate on every layer can wipe out the pretrained filters before the head has learned anything.
- `d`: ImageNet's classes are not your classes. Train a head for your labels.
- `a|r2`: Right plan, wrong reason. It is about reusing learned features with little data.

B24-L5 exercise 2

Hints. (1) Which parameters exist after the loop has run? (2) What is requires_grad on a freshly built layer? (3) 512 inputs, 3 outputs.

Answer.

```
1 ['fc.weight', 'fc.bias'] 1539
```

The freeze touched only the old parameters. The new head has 512 × 3 weights and 3 biases, all trainable: 1,539.

Common wrong answers.

- `[] 0`: The new Linear layer is created after the freeze, and new layers have requires_grad=True.
- `['fc.weight', 'fc.bias'] 1536`: The bias counts too: 512 * 3 + 3.

B24-L5 exercise 3

Hints. (1) Which flag tells autograd to skip a tensor? (2) The new head receives what the old head received. (3) `requires_grad` and `in_features`.

Answer.

```python
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 3)
```

`requires_grad = False` freezes each weight, and `model.fc.in_features` (512) sizes the new head.

Common wrong answers.

- g1 = `requires_grad`; g2 = `out_features`: out_features is 1000, the ImageNet classes. The new head reads the same 512 inputs: in_features.

B24-L5 exercise 4

Hints. (1) Check `image.ndim`, the two lengths and `(std <= 0).any()`. (2) Reshape the statistics to `(C, 1, 1)` with `[:, None, None]`. (3) `(image - mean[:, None, None]) / std[:, None, None]`.

Answer.

```python
import numpy as np


def normalise_chw(image: np.ndarray, mean: np.ndarray, std: np.ndarray) -> np.ndarray:
    """Normalise a (C, H, W) image channel by channel.

    Args:
        image: Float image in [0, 1], channels first.
        mean: One mean per channel.
        std: One positive standard deviation per channel.

    Returns:
        The normalised image, same shape.

    Raises:
        ValueError: If the shapes do not match or a std is not positive.
    """
    if image.ndim != 3 or len(mean) != image.shape[0] or len(std) != image.shape[0]:
        raise ValueError(f"need (C, H, W) and C statistics, got {image.shape}, {len(mean)}, {len(std)}")
    if (std <= 0).any():
        raise ValueError("every standard deviation must be positive")
    return (image - mean[:, None, None]) / std[:, None, None]
```

Reshaped to `(C, 1, 1)`, each statistic broadcasts over its own channel's height and width. Without it, a length-3 array would line up with the last axis, the width.

B24-L5 exercise 5

Hints. (1) Which part has random weights at the start? (2) What inputs did the backbone learn from? (3) Fine-tune gently, late layers first.

Answer.

- Train the head first, so random head weights do not send large gradients into good filters.
- Use the preprocessing the backbone was trained with.
- Then fine-tune the last block gently, with its own smaller learning rate in a parameter group.

Head first, the backbone's own preprocessing, then a gentle fine-tune of the last block.

Common wrong answers.

- start = `all`: A random head sends large, noisy gradients into every layer and can damage the pretrained filters.
- start = `scratch`: 2,000 photos are few for 11.7 million parameters. Start from the pretrained weights.
- prep = `own`: The filters were trained on inputs normalised with ImageNet's statistics. Keep them.
- prep = `none`: The backbone expects normalised floats at 224 by 224.
- next = `all_high`: A high rate on every layer can undo what pretraining learned. Unfreeze the last block with a smaller rate.

B24-L5 exercise 6

Hints. (1) Which parameters does the optimiser hold? (2) Did the new head exist when the optimiser was built? (3) Replace the head first, then build the optimiser.

Answer. The bug is on line 8.

Fixed version:

```
1 import torch
2 from torch import nn
3 from torchvision.models import ResNet18_Weights, resnet18
4
5 model = resnet18(weights=ResNet18_Weights.DEFAULT)
6 for param in model.parameters():
7     param.requires_grad = False
8 model.fc = nn.Linear(model.fc.in_features, 4)
9 optimizer = torch.optim.AdamW((p for p in model.parameters() if
    p.requires_grad), lr=1e-3)
```

The optimiser holds the old, frozen parameters, including the replaced 1,000-class head. The new head is not in it, so `step()` never updates it. Build the optimiser after replacing `model.fc`, from trainable parameters only.

Common wrong answers.

- Line 7: Freezing the backbone is right. The question is what the optimiser holds.
- Line 9: The new head is built correctly. The optimiser was created before it existed.

B24-L6 Evaluating image classifiers, per class and on honest splits

B24-L6 exercise 1

Hints. (1) What accuracy does a model reach if it never predicts the small class? (2) Which numbers are per class? (3) Recall from the confusion matrix.

Answer. a) The confusion matrix and the recall of each class Because: r1) A model can miss the small class entirely and still score 0.95. Recall per class shows it.

Accuracy averages over images, so a small class can fail completely inside a high score. Per-class recall shows each class.

Common wrong answers.

- `b`: Always predicting the other classes can reach 0.95 with a recall of 0 for the small class.
- `c`: The loss curve shows how training went, not which classes fail on validation.
- `d`: Training accuracy measures memorisation, not performance on new images.
- `a|r2`: Right check, wrong reason. It is about finding classes that a high average hides.

B24-L6 exercise 2

Hints. (1) Row = true class, column = predicted class. (2) True 2 was predicted once as 2 and once as 0. (3) Recall: diagonal over row sums.

Answer.

```
1 [[1, 1, 0], [0, 2, 0], [1, 0, 1]] [0.5, 1.0, 0.5]
```

Class 0 has one hit of two, class 1 two of two, class 2 one of two. Dividing the diagonal by row totals gives recall.

Common wrong answers.

- `[[1, 0, 1], [1, 2, 0], [0, 0, 1]] [0.5, 1.0, 0.5]`: Rows are true classes and columns predicted. That is the transpose.
- `[[1, 1, 0], [0, 2, 0], [1, 0, 1]] [0.5, 0.6667, 1.0]`: Dividing by column totals gives precision. Recall divides by row totals.

B24-L6 exercise 3

Hints. (1) Which block turns off gradient tracking? (2) Which method returns the position of the largest logit? (3) `no_grad` and `argmax`.

Answer.

```
model.eval()
with torch.no_grad():
    preds = model(X).argmax(dim=1)
```

`torch.no_grad()` skips the graph, and `argmax(dim=1)` returns the index of the largest logit in each row: the predicted class.

Common wrong answers.

- g1 = `no_grad`; g2 = `max`: max returns values and indices together. argmax returns the class index alone.

B24-L6 exercise 4

Hints. (1) Check the lengths, then the range of every label. (2) Start from `np.zeros((n_classes, n_classes), dtype=np.int64)`. (3) For each pair, `cm[t, p] += 1`.

Answer.

```
import numpy as np


def confusion(y_true: np.ndarray, y_pred: np.ndarray, n_classes: int) -> np.ndarray:
    """Confusion matrix: true classes in rows, predicted classes in columns.

    Args:
        y_true: True class indices.
        y_pred: Predicted class indices.
        n_classes: Number of classes.

    Returns:
        Integer counts, shape (n_classes, n_classes).

```

```
    Raises:
        ValueError: If the lengths differ or a label is out of range.
    """
    if len(y_true) != len(y_pred):
        raise ValueError(f"{len(y_true)} true labels and {len(y_pred)}
predictions")
    labels = np.concatenate([y_true, y_pred])
    if len(labels) and (labels.min() < 0 or labels.max() >= n_classes):
        raise ValueError(f"labels must lie in 0 to {n_classes - 1}")
    cm = np.zeros((n_classes, n_classes), dtype=np.int64)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm
```

Each (true, predicted) pair adds one to row `t`, column `p`. Sizing from `n_classes` keeps the matrix the same shape for every batch, even when a class is missing.

B24-L6 exercise 5

Hints. (1) Check `cm.shape[0] == cm.shape[1]`. (2) Row totals: `cm.sum(axis=1)`. (3) Divide by `np.maximum(totals, 1)` inside `np.where(totals > 0, ..., np.nan)`.

Answer.

```
import numpy as np


def recall_by_class(cm: np.ndarray) -> np.ndarray:
    """Recall per class from a confusion matrix with true classes in rows.

    Args:
        cm: Square matrix of counts.

    Returns:
        Recall per class, NaN where a class has no true examples.

    Raises:
        ValueError: If cm is not square.
    """
    if cm.ndim != 2 or cm.shape[0] != cm.shape[1]:
        raise ValueError(f"need a square matrix, got {cm.shape}")
    totals = cm.sum(axis=1)
    return np.where(totals > 0, cm.diagonal() / np.maximum(totals, 1),
np.nan)
```

Dividing by at least 1 avoids the warning, and `np.where` swaps in NaN for empty classes, so a missing class never reads as perfect or as zero.

B24-L6 exercise 6

Hints. (1) What does the permutation shuffle: photos or shelves? (2) Where do the other four photos of a validation shelf end up? (3) Shuffle np.unique(shelf_ids) instead.

Answer. The bug is on line 5.

Fixed version:

```
import numpy as np


def split_photos(shelf_ids, val_share, rng):
    shelves = rng.permutation(np.unique(shelf_ids))
    n_val = round(len(shelves) * val_share)
    in_val = np.isin(shelf_ids, shelves[:n_val])
    return np.flatnonzero(~in_val), np.flatnonzero(in_val)
```

Shuffling photos sends most bursts to both sides, so the model is graded on near-copies of training photos. Shuffle the distinct shelves and keep each shelf's photos on one side.

Common wrong answers.

- Line 6: The share is fine. The split is cut along the wrong unit.
- Line 7: Returning two sets of positions is right. They were drawn photo by photo.